Use Cases

for

<Project>

Version 1.0 approved

Prepared by <author>

<organization>

<date created>

Revision History

Name	Date	Reason For Changes	Version

Guidance for Use Case Template

Document each use case using the template shown in the Appendix. This section provides a description of each section in the use case template.

1. Use Case Identification

1.1. Use Case ID

Give each use case a unique numeric identifier, in hierarchical form: X.Y. Related use cases can be grouped in the hierarchy. Functional requirements can be traced back to a labeled use case.

1.2. Use Case Name

State a concise, results-oriented name for the use case. These reflect the tasks the user needs to be able to accomplish using the system. Include an action verb and a noun. Some examples:

- View part number information.
- Manually mark hypertext source and establish link to target.
- Place an order for a CD with the updated software version.

1.3. Use Case History

1.3.1. Created By

Supply the name of the person who initially documented this use case.

1.3.2. Date Created

Enter the date on which the use case was initially documented.

1.3.3. Last Updated By

Supply the name of the person who performed the most recent update to the use case description.

1.3.4. Date Last Updated

Enter the date on which the use case was most recently updated.

2. Use Case Definition

2.1. Actor

An actor is a person or other entity external to the software system being specified who interacts with the system and performs use cases to accomplish tasks. Different actors often correspond to different user classes, or roles, identified from the customer community that will use the product. Name the actor(s) that will be performing this use case.

2.2. Description

Provide a brief description of the reason for and outcome of this use case, or a high-level description of the sequence of actions and the outcome of executing the use case.

2.3. Preconditions

List any activities that must take place, or any conditions that must be true, before the use case can be started. Number each precondition. Examples:

1. User's identity has been authenticated.
2. User's computer has sufficient free memory available to launch task.

2.4. Postconditions

Describe the state of the system at the conclusion of the use case execution. Number each postcondition. Examples:

1. Document contains only valid SGML tags.
2. Price of item in database has been updated with new value.

2.5. Priority

Indicate the relative priority of implementing the functionality required to allow this use case to be executed. The priority scheme used must be the same as that used in the software requirements specification.

2.6. Frequency of Use

Estimate the number of times this use case will be performed by the actors per some appropriate unit of time.

2.7. Flow of Events

Provide a detailed description of the user actions and system responses that will take place during execution of the use case under normal, expected conditions. This dialog sequence will ultimately lead to accomplishing the goal stated in the use case name and description. This description may be written as an answer to the hypothetical question, "How do I <accomplish the task stated in the use case name>?" This is best done as a numbered list of actions performed by the actor, alternating with responses provided by the system.

2.8. Alternative Flows

Document other, legitimate usage scenarios that can take place within this use case separately in this section. State the alternative course, and describe any differences in the sequence of steps that take place. Number each alternative course using the Use Case ID as a prefix, followed by "AC" to indicate "Alternative Course". Example: X.Y.AC.1.

2.9. Exceptions

Describe any anticipated error conditions that could occur during execution of the use case, and define how the system is to respond to those conditions. Also, describe how the system is to respond if the use case execution fails for some unanticipated reason. Number each exception using the Use Case ID as a prefix, followed by "EX" to indicate "Exception". Example: X.Y.EX.1.

2.10. Includes

List any other use cases that are included ("called") by this use case. Common functionality that appears in multiple use cases can be split out into a separate use case that is included by the ones that need that common functionality.

2.11. Special Requirements

Identify any additional requirements, such as nonfunctional requirements, for the use case that may need to be addressed during design or implementation. These may include performance requirements or other quality attributes.

2.12. Assumptions

List any assumptions that were made in the analysis that led to accepting this use case into the product description and writing the use case description.

2.13. Notes and Issues

List any additional comments about this use case or any remaining open issues or TBDs (To Be Determineds) that must be resolved. Identify who will resolve each issue, the due date, and what the resolution ultimately is.

Use Case Template

Use Case ID:	UC.2		
Use Case Name:	Retrieve, View, and Filter SkillsFuture Courses for Selected Industry		
Created By:	Gan Sui	Last Updated By:	Gan Sui
Date Created:	21/2/26	Date Last Updated:	21/2/26

Actor:	Job Seeker
Description:	This use case allows a Job Seeker to retrieve SkillsFuture course data and view at least 10 relevant courses for a selected industry. The user can apply filters (price, duration, language), see the count of matching courses, and open a course link to SkillsFuture. This use case is initiated from UC1 after the user selects an industry.
Preconditions:	1. UC.1 is active and the user has selected an industry from the Top 10 list. 2. System has access to the SkillsFuture course directory Excel via API. 3. User has access to the website via supported browser (Chrome/Firefox/Safari).
Postconditions:	1. System displays relevant courses (>=10 if available) for the selected industry. 2. Applied filters and matching course count are shown. 3. When a course link is clicked, the user is redirected to SkillsFuture (new tab or same tab per design).
Priority:	High

Frequency of Use:	High
Flow of Events:	1. The Job Seeker selects an industry from the Top 10 industry list displayed in UC.1. 2. The system retrieves the SkillsFuture course directory Excel file from the API, extracts the required course fields, and matches courses to the selected industry using keyword matching on the course title and course rating fields. 3. The system displays at least 10 relevant courses for the selected industry (if available) and shows each course's reference number, title, full course fee (SGD), number of hours, course rating stars, job impact rating stars, and a clickable link to the SkillsFuture website. 4. The Job Seeker applies filters for price range, course duration, and course language. 5. The system updates the displayed course list based on the selected filters and shows the number of courses that match the current filters. 6. The Job Seeker clicks a course from the displayed list. 7. The system redirects the Job Seeker to the selected course page on the SkillsFuture website.

Alternative Flows:	UC.2.A1: Fewer than 10 courses are available 1. The system displays all matching courses that are available and indicates that fewer than 10 relevant courses were found for the selected industry. UC.2.A2: No courses match after filters are applied 1.The Job Seeker applies one or more filters for price range, duration, or language. 2. The system determines that no courses match the selected filters. 3. The system displays a “No courses found” message and prompts the Job Seeker to remove or adjust the filters. UC.2.A3: The Job Seeker changes the selected industry 1. The Job Seeker selects a different industry from the Top 10 list after previously selecting one. 2. The system updates the selected industry state and refreshes the displayed courses based on the newly selected industry. 3. The system resets the course list to the newly matched results and updates the course count for the current filter settings.

Exceptions:	UC.2.E1: SkillsFuture course data retrieval fails 1. The system fails to retrieve the SkillsFuture course directory Excel file from the API. 2. The system displays an error message stating that course data is temporarily unavailable and provides a retry option. UC.2.E2: SkillsFuture course file format is invalid or missing required fields 1. The system retrieves the course file but detects missing or invalid fields required for matching or display. 2. The system displays an error message stating that course data cannot be processed and logs the issue for maintainers. UC.2.E3: Course URL is missing or invalid 1. The system detects that a displayed course does not contain a valid SkillsFuture redirect link. 2. The system disables the link for that course and displays a message stating that the course link is unavailable.
Includes:	UC1: Job Seeker must select an industry from the Top 10 industry list before course retrieval can occur.

Special Requirements:	1. The system must display course search results within 3 seconds after the Job Seeker selects an industry. 2. The system must update the displayed course list within 1 second after the Job Seeker applies any filter. 3. The system must display the number of courses matching the current filters whenever filters are applied or changed. 4. The system must work correctly on modern browsers, including Chrome, Firefox, and Safari. 5. The system must follow kebab-case naming conventions for CSS classes and snake_case naming conventions for templates, variables, functions, files, and database columns.
Assumptions:	1. The SkillsFuture course directory contains sufficient information to display the required course fields, including fee, duration, ratings, language, and a SkillsFuture redirect URL. 2. Keyword matching is an acceptable method for identifying relevant courses for an industry in this project's scope, even if relevance is not perfect. 3. The course directory file retrieved via API is updated regularly enough that daily refresh of cached data is sufficient for the website.
Notes and Issues:	1. The keyword matching rules clearly, including whether industry synonyms are used and whether ratings should affect ranking. 2. The filters should reset or persist when the Job Seeker changes the selected industry. 3. The team should implement caching of course data (with daily refresh) to meet performance requirements for page load and filtering responsiveness.